Thapar Institute of Engineering and Technology

Parallel and Distributed Computing

LAB Assignment 7

Submitted by :

Bhavish Pushkarna

102303279

3C22

Submitted to

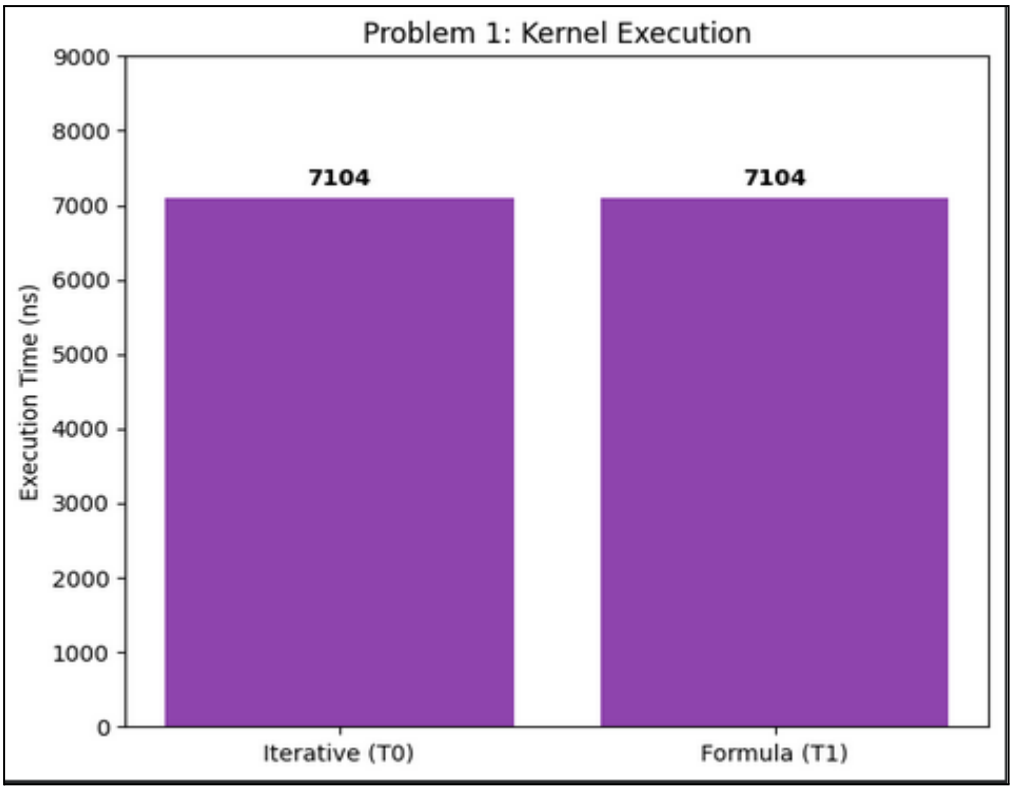
Dr. Saif Nalband

Question 1: Task Divergence Analysis:

Objective: Compare iterative vs. formula-based summation in a single kernel.

Performance Data:

Metric	Thread 0 (Iterative)	Thread 1 (Formula)
Result	524,800	524,800
Kernel Execution Time	7,104 ns	7,104 ns



Inference

- **Warp Serialization:**

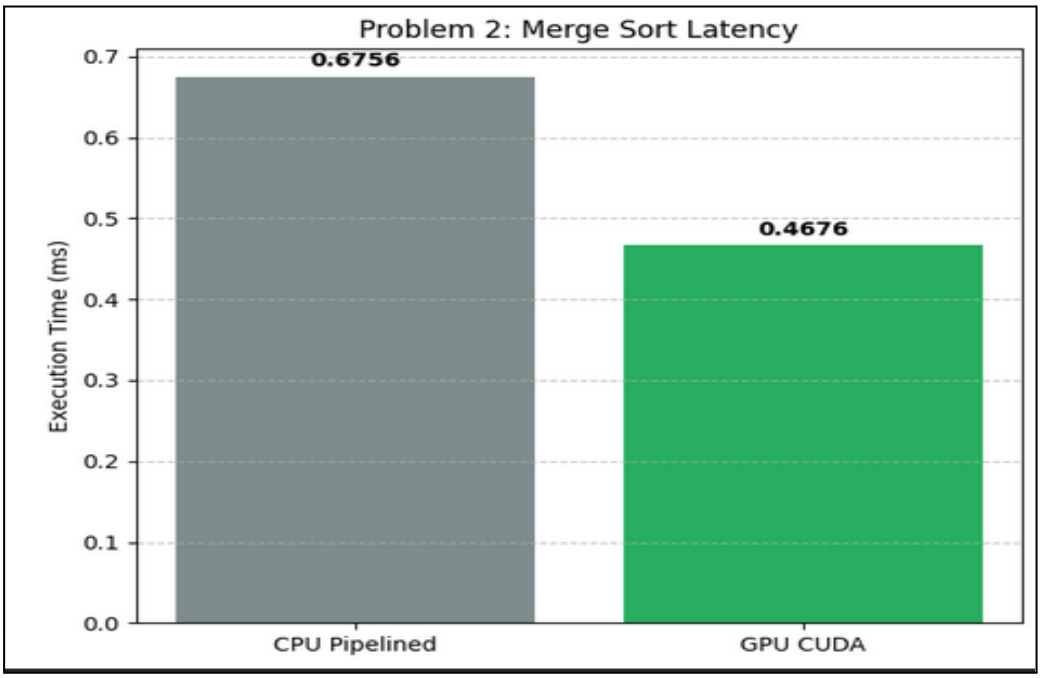
  Even though the formula is mathematically O(1), the profiler shows both threads are tied to the same taskKernel execution. Because these threads reside in the same warp, the GPU executes the divergent paths sequentially. Thread 1 (Formula) effectively "waits" for Thread 0 (Iterative) to complete its loop before the warp can retire.

Q2: Sorting Performance (N=1000)

Objective: Compare CPU Merge Sort against GPU Parallel Merge Sort.

Performance Data :

Implementation	Execution Time (ms)	Speedup
CPU Merge Sort	0.675591 ms	1.00x (Baseline)
GPU Merge Sort	0.467648 ms	1.44x



Inference:

- **Efficiency:**

  You achieved a **1.44x speedup**. However, the Nsight report shows cudaMalloc took **128 ms** i.e 95.9% of total API time.

- **Small Dataset Bottleneck:**

  For N=1000, the actual computation on the GPU (192,801 ns) is fast, but when you include the overhead of memory allocation and data transfer, the GPU is less efficient than the CPU for such a small workload. Parallelism only “pays off” when the computation time significantly exceeds the overhead of moving data to the device.

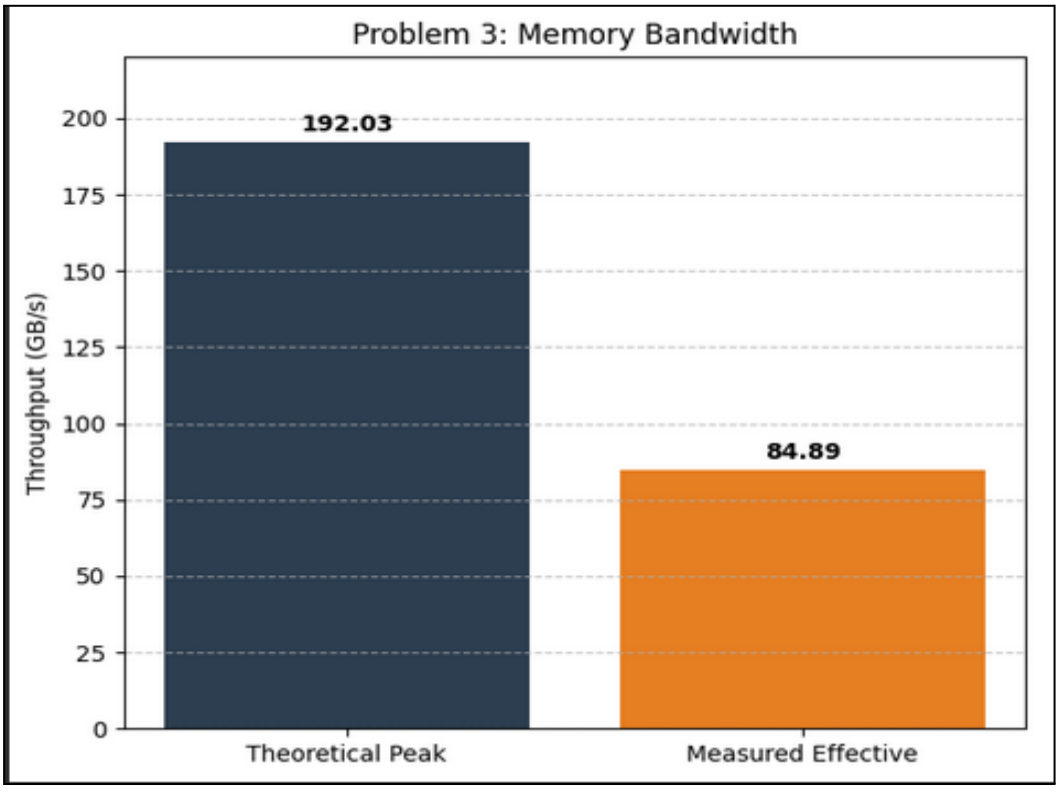
Q3: Memory Bandwidth Profiling

Objective: Measure effective throughput of a Vector Addition kernel.

Performance Data:

Metric	Value
Kernel Execution Time	0.15 ms (82,913 ns)
Theoretical Bandwidth	192.03 GB/s
Measured Bandwidth	84.89 GB/s
Bus Efficiency	**44.21%**

Graphical Representation:



Inference:

- **Memory Bound Kernel:**

  The efficiency of 44.21% is typical for a basic Vector Addition kernel.

- **Why the Gap?**

  The theoretical bandwidth assumes ideal, continuous streaming. In practice, the measured bandwidth is lower due to **Memory Latency** i.e the time taken to open/close rows in DRAM and **Instruction Overhead**.

- **Static Symbols:** Using “cudaMemcpyToSymbol” seen in your logs taking 130ms ensures the data is in global memory, but the bus is still the primary bottleneck.